

ALGORITHMS AND DATA STRUCTURES
(25COP923)

Sample Practice Exam Paper

2 hours

This examination is to take place in-person at a central University venue under exam conditions. The standard length of time for this paper is **2 hours**.

You will not be able to leave the exam hall for the first 30 or final 15 minutes of your exam.

Your invigilator will collect your exam paper when you have finished.

Help during the exam

Invigilators are not able to answer queries about the content of your exam paper. Instead, please make a note of your query in your answer script to be considered during the marking process.

If you feel unwell, please raise your hand so that an invigilator can assist you.

You may **not** use a calculator for this exam.

Write your answer for every question in the appropriate space. Indicate within that space if you give additional parts of your answer elsewhere.

1. Definitions. Provide informal definitions of the concepts below. [10]

- (a) What is a graph? Provide an informal definition and explain the difference between directed and undirected graphs. [5]

Solution: A graph is a mathematical structure used to model pairwise relationships between objects. It consists of a set of vertices (or nodes) and a set of edges (or links) that connect pairs of vertices.

In a directed graph (or digraph), the edges have a direction, meaning they go from one vertex to another, indicating a one-way relationship. In contrast, in an undirected graph, the edges do not have a direction, indicating a two-way relationship between the connected vertices. [5]

- (b) What is an hard problem? Explain the difference between P and NP problems. [5]

Solution: A hard problem is a computational problem that is difficult to solve efficiently, often requiring significant time or resources as the size of the input grows.

- P problems are those that can be solved in polynomial time.
- NP problems are those for which a proposed solution can be verified in polynomial time.

[5]

2. Algorithm analysis. [15]

What is the worst-case time complexity of the following algorithm in big-Oh notation? What task is performed by the algorithm? How is that task achieved? (Write the respective answers into the boxes.)

Procedure(A, n)

```

1  for  $i \leftarrow 1$  to  $n - 1$  do
2      for  $j \leftarrow i + 1$  to  $n$ 
3          if  $A[i] = A[j]$  then
4              return true
5  return false

```

Complexity?	Solution: $O(n^2)$
Task performed?	Solution: Detect whether the array A contains any duplicate elements (returns true if a duplicate exists, otherwise false)
How achieved?	Solution: The algorithm examines every unordered pair of elements in the array by using a nested loop: for each index i it compares $A[i]$ with every later element $A[j]$ (for $j = i+1..n$). If any pair is equal, the algorithm immediately returns true ; if no equal pair is found after all comparisons, it returns false . In the worst case (no duplicates), it performs $\Theta(n^2)$ comparisons, hence the time complexity is $O(n^2)$.

3. Data structures.

[20]

(a) Consider the singly linked list shown below:



(i) Fill out the following table.

[8]

Traversal	Sequence of nodes visited
Sequence of nodes visited in a head-to-tail traversal	
Sequence of nodes visited in a search for value 9	
Number of elements in the list	
Value of the head of the list	
Value of the tail of the list	

Solution: Forward traversal: 4, 7, 1, 9, 5

Search for 9: 4, 7, 1, 9 (found)

Node count: 5 nodes; Head: 4; Tail: 5

(b) Suppose the linked list data structure is implemented using nodes containing:

- a data field (`value`)
- a pointer to the next node (`next`)

Write the pseudocode of the recursive primitives for the following linked list operations.

Use the following function signatures:

[12]

(i) **Forward Traversal** (Head $\rightarrow$ Tail)Traverse(*node*)**Solution:**

```

1  if node = NIL then
2      return
3  visit(node.value)
4  Traverse(node.next)
  
```

(ii) Search for a value

`Search(node, target)`

Solution:

```
1  if node = NIL then  
2      return false  
3  if node.value = target then  
4      return true  
5  return Search(node.next, target)
```

(iii) Count the nodes in the list

`CountNodes(node)`

Solution:

```
1  if node = NIL then  
2      return 0  
3  return 1 + CountNodes(node.next)
```

4. Graphs. [35]

A large logistics company is planning an upgrade of its delivery drone network. The company operates many one-way flight routes between distribution hubs. Each route has an associated travel time that depends on prevailing winds.

To guarantee fast deliveries, management wants to mark certain routes as *priority air corridors*. Drones travelling along a priority corridor experience reduced delays, making these routes effectively “fast lanes”.

The company imposes the following rules:

- Each existing flight route must remain available, either as a normal route or as a faster priority corridor.
- Priority corridors follow the direction of the underlying flight route.
- For a designated central hub s , every other hub must be reachable from s using only priority corridors.
- The total travel time of all priority corridors must be as small as possible.

The aim is to build the minimal set of directed priority routes that guarantees fast reachability from the single source hub s .

- (a) Explain how this problem can be described and formalised in terms of graphs. [10]

Solution: We model the drone network as a weighted directed graph:

- each hub is a vertex,
- each one-way flight route is a directed edge,
- the weight of each edge is the travel time for that route.

The requirement that every hub must be reachable from the central hub s via priority corridors means that the priority must be decided upon the shortest distances from s .

Minimising the total travel time corresponds to selecting, for each reachable vertex v , the directed edge that provides the shortest-path distance from s to v .

Therefore, the task is equivalent to computing the *shortest paths* from s to every other vertex in a directed weighted graph.

- (b) Design an algorithm that determines which routes should be upgraded to priority air corridors. Describe it in words and provide high-level pseudocode. State which known algorithm your solution is based on. [15]

Solution: We solve the problem using **Dijkstra's algorithm** for single-source shortest paths in directed graphs with non-negative edge weights.

Algorithm outline:

1. Take the designated central hub s as the source.

2. Run Dijkstra's algorithm to compute the shortest travel-time distance from s to every other hub.
3. For each hub $v \neq s$, identify the incoming edge $(p(v), v)$ that last relaxed its distance.
4. Mark each such edge as a *priority corridor*.

The resulting set of edges forms the directed shortest-path tree rooted at s and provides minimum total travel time while maintaining reachability from s .

PriorityCorridors($G = (V, E), s$)

```

1  for each  $v \in V$  do
2       $\text{dist}[v] \leftarrow \infty$ 
3       $\text{parent}[v] \leftarrow \text{null}$ 
4   $\text{dist}[s] \leftarrow 0$ 
5  PQ  $\leftarrow$  priority queue containing  $(s, 0)$ 
6  while PQ not empty do
7       $u \leftarrow \text{PQ.extractMin}()$ 
8      for each directed edge  $(u, w)$  with weight  $t$  do
9          if  $\text{dist}[u] + t < \text{dist}[w]$  then
10              $\text{dist}[w] \leftarrow \text{dist}[u] + t$ 
11              $\text{parent}[w] \leftarrow u$ 
12             PQ.insertOrDecreaseKey( $w, \text{dist}[w]$ )
13 Corridors  $\leftarrow \{(\text{parent}[v], v) \mid v \neq s\}$ 
14 return Corridors

```

- (c) Discuss a suitable representation of the directed graph and an implementation of your algorithm. Mention any auxiliary data structures and estimate the running time of your approach. [10]

Solution: A directed graph with weighted edges is efficiently represented using:

- an adjacency list: for each vertex, a list of outgoing edges and their weights,
- arrays $\text{dist}[]$ and $\text{parent}[]$ to store shortest-path information,
- a min-heap priority queue (e.g. binary heap) for efficiently selecting the next vertex with the smallest tentative distance.

With an adjacency list and a binary heap, Dijkstra's algorithm runs in: $O((|V| + |E|) \log |V|)$ time.

Extracting the final set of priority corridors from $\text{parent}[]$ takes an additional $O(|V|)$ time.

5. Kruskal's Algorithm.

[20]

(a) Let $G = (V, E)$ be a graph and w a function assigning a weight to each edge of G .

Define:

- what is a *spanning tree* of G ;
- what is a *minimum spanning tree* (MST) of G .

[5]

Solution: A *spanning tree* of a connected graph G is a subgraph that:

- contains all vertices of G ,
- is a tree (connected and acyclic).

An *MST* is a spanning tree whose total weight $\sum_{e \in T} w(e)$ is minimal among all spanning trees of G .

(b) In the following questions refer to the Kruskal's algorithm.

(i) Briefly state the problem solved by Kruskal's algorithm, indicating:

- the input to the algorithm;
- the output of the algorithm.

[5]

Solution: Kruskal's algorithm solves the minimum spanning tree problem for a connected, weighted, undirected graph.

- Input: A connected undirected graph $G = (V, E)$ with edge weights w .
- Output: An MST of G .

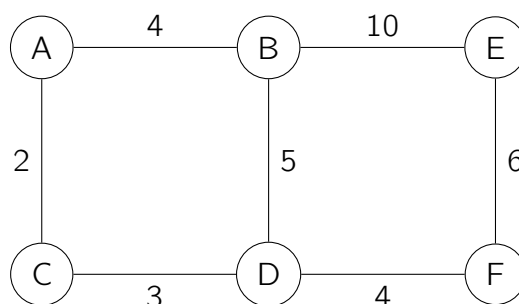
(ii) Here is a simple, non-optimised version of Kruskal's algorithm:

Kruskal-simple($G = (V, E), w$)

```

1  sort edges in  $E$  by non-decreasing weight
2   $MST \leftarrow \emptyset$ 
3  for each edge  $(u, v)$  in sorted order do
4      if adding  $(u, v)$  to  $MST$  does not create a cycle then
5          add  $(u, v)$  to  $MST$ 
6  return  $MST$ 
```

Consider the following graph:



Run the simple Kruskal algorithm on this graph. Complete the table below after each step, indicating:

- the edge currently being considered (sorted by weight);
- whether the algorithm chooses it;
- the current set of edges in the MST.

[10]

Step	Edge	Taken?	Current MST edges
1			
2			
3			
4			
5			
6			
7			

Solution: Sorted edges:

$(A, C : 2)$, $(C, D : 3)$, $(A, B : 4)$, $(D, F : 4)$, $(B, D : 5)$, $(E, F : 6)$, $(B, E : 10)$.

In each step, we simply check whether adding the edge forms a cycle in the current partial graph.

Step	Edge	Taken?	Current MST edges
1	A–C (2)	Yes	{A–C}
2	C–D (3)	Yes	{A–C, C–D}
3	A–B (4)	Yes	{A–C, C–D, A–B}
4	D–F (4)	Yes	{A–C, C–D, A–B, D–F}
5	B–D (5)	No	(creates a cycle A–C–D–B–A)
6	E–F (6)	Yes	{A–C, C–D, A–B, D–F, E–F}
7	B–E (10)	No	already connected through F

Final MST total weight:

$$2 + 3 + 4 + 4 + 6 = 19.$$